

ENTWICKLUNGS- UMGEBUNG



ENTSCHEIDUNG ZUR ENTWICKLUNGSUMGEBUNG UND GENUTZTEN TOOLS FÜR DIE ERSTELLUNG DER ENTERTAINMENT-ERWEITERUNG FÜR IFE

Stand: 02.04.26

Auftraggeber: Novaris Cabin Systems GmbH

Friedrich-List-Platz 1

01069 Dresden

Ansprechpartner*in: Lea Wagner

E-Mail: lwagner@novaris-cabinystems.de

Telefon: 0351 4620

Auftragnehmer: Gervithrall Systems GmbH

Perlickstraße 1

04103 Leipzig

Ansprechpartner*in: Lucas Rumann

E-Mail: lucasr@gervithrall-systems.de

Telefon: 0351 6482642

Versionierung

Kriterium	Git	Subversion (SVN)	Mercurial (hg)
Benutzerfreundlichkeit (x0.75)	6: Viele Befehle und komplexe Konzepte machen den Einstieg schwierig	8: Einfaches, zentrales System, leicht verständlich für Anfänger	7: Ähnlich wie Git, aber mit klarerer und einfacherer Bedienung
Performance und Architektur (x1.75)	9: Sehr schnell, da lokal bzw. dezentral gearbeitet wird, und damit auch sehr robust	6: Langsamer, da viele Aktionen über den Server laufen, von dem man Abhängig ist	8: Ebenfalls schnell durch dezentrale und unabhängige Struktur
Verbreitung (x1.0)	10: Globaler Standard, riesige Community und viele Ressourcen	7: Noch verbreitet, aber zunehmend durch Git ersetzt	5: Deutlich kleinere Community und weniger verbreitet als Git oder SVN
Branching und Merging (x1.5)	10: Sehr leistungsfähig und flexibel bei Branches und Merges	5: Branching möglich, aber umständlich und fehleranfalliger	8: Gute Unterstützung, aber etwas weniger flexibel als Git
Lernkurve (x0.5)	5: Steile Lernkurve, viele Konzepte müssen verstanden werden, für Einsteiger sehr unübersichtlich/komplex	8: Schnell erlernbar durch einfache Struktur	7: Leichter als Git, aber trotzdem gewisse Komplexität
Tools und Integration (x1.25)	10: Sehr viele Tools, IDE-Integrationen und Plattformen (z. B. GitHub)	7: Gute Integration, aber weniger modern und umfangreich	6: Aufgrund geringer Verbreitung weniger Tools und Integrationen verfügbar
Gesamt	60.25 / 67.5 = 89.3%	43.75 / 67.5 = 64.8%	47.25 / 67.5 = 70%

Fazit: Unsere Wahl fällt aufgrund der technischen Leistungsfähigkeit auf Git. Unter anderem da das Team mit Git bereits eingearbeitet ist, und somit die Einstiegshürden für uns keine Rolle mehr spielen.

UML-Tool

Kriterium	Papyrus (Eclipse-Plugin)	StarUML	PlantUML
Kosten / Lizenz (x1.25)	10: Kostenlos und Open Source	5: Kostenpflichtig, Testversion verfügbar	10: Kostenlos und Open Source
Plattformen (x1.0)	9: Läuft überall mit Eclipse	9: Win / macOS / Linux	10: Plattformunabhängig
UML2-Unterstützung (x1.5)	10: Vollständige UML2-Unterstützung	8: Gute UML-Unterstützung	8: Gute UML-Unterstützung
Code-Generierung (x0.75)	7: Möglich über Plugins	6: Eingeschränkt	4: Kaum vorhanden
Integration in IDE (x1.5)	10: Direkt in Eclipse integriert	6: Eigenständiges Tool	9: Sehr gut mit VS Code integrierbar
Bedienbarkeit (x1.0)	6: Etwas komplex und altmodisch	8: Moderne und intuitive Oberfläche	7: Textbasiert, erfordert Eingewöhnung
Wartbarkeit / Versionierung (x1.25)	7: Eingeschränkt über Eclipse	6: Schwer versionierbar	10: Perfekt für Git geeignet (textbasiert)
Gesamt	71.5 / 82.5 = 86.7%	56.25 / 82.5 = 68.2%	70.5 / 82.5 = 85.5%

Fazit: PlantUML wird im Projekt bevorzugt eingesetzt, da es eine sehr gute Integration in den bestehenden Entwicklungsworkflow bietet, besonders effizient für Versionierung über Git ist und eine schnelle, reproduzierbare Erstellung von UML-Diagrammen ermöglicht.

Build-Tool

Kriterium	Ant	Maven	Gradle
Konfigurationsaufwand (x1.75)	2: Extrem hoch, die Schritte Kompilieren, Kopieren und Löschen müssen manuell definiert werden.	9: Sehr gering, mit minimalem manuellem Setup	7: Moderat, die DSL benötigt mehr Einarbeitung als Mavens XML Standard
Build Performance (x1.5)	5: Langsam	7: Solide, parallele Builds oft komplex zu konfigurieren	10: Überragend, mit sehr effizientem Build Cache
Dependency Management (x1.25)	3: Ursprünglich nicht vorhanden, kann umständlich nachgerüstet werden	9: Exzellent, das Maven-Repository ist der Industriestandard	10: Hochmodern, unterstützt Maven und Ivy Repositories mit einer besseren Konfliktlösung
Lesbarkeit und Wartbarkeit (x1.5)	4: XML-Dateien wachsen bei größeren Projekten stark und sind schwer zu pflegen	7: Gut lesbar durch strukturiertes XML, kann bei vielen Plugins leicht unübersichtlich werden	8: Sehr kompakt und leserlich, erfordert allerdings Verständnis der Skriptsprache
Standardisierung (x1.0)	4: Kaum Standards	10: Goldstandard, jedes Projekt folgt der gleichen Struktur	9: Stark standardisiert, erlaubt allerdings einige Freiheiten
Skalierbarkeit (x0.75)	4: Stößt bei Multi-Modul Projekten schnell an Grenzen	7: Gut für große Projekte, kann aber auch bei vielen Modulen in der Performance einbrechen	10: Speziell für riesige Projekte entwickelt
Gesamt	27.75 / 77.5 = 35.8%	63.25 / 77.5 = 81.6%	68.25 / 77.5 = 88.1%

Fazit: Auch wenn Gradle mit einem knappen Vorsprung Maven schlägt, fällt die interne Wahl für dieses Projekt auf Maven. Dies ist nach wie vor die sicherste Wahl mit seiner Standardstruktur für unser geplantes Projekt.

UI-Prototyping-Tool

Kriterium	Figma	Balsamiq	Adobe XD
Kosten / Lizenz (x1.25)	9: Kostenloser Free-Plan verfügbar	5: Kostenpflichtig	7: Teil der Adobe Creative Cloud
Zugriff / Installation (x1.0)	10: Browser + Desktop-App	7: Nur Desktop	8: Desktop + Cloud-Anbindung
Zusammenarbeit (x1.5)	10: Echtzeit-Kollaboration	4: Kaum vorhanden	8: Gute Cloud-Zusammenarbeit
Bedienbarkeit (x1.0)	9: Intuitiv und modern	9: Sehr einfach	9: Sehr intuitiv
Funktionsumfang (x1.5)	10: Sehr umfangreich	6: Stark reduziert	9: Umfangreich
Export / Dokumentation (x1.0)	9: Sehr guter Export	8: Gute Exportmöglichkeiten	9: Sehr guter Export
Performance (x1.0)	9: Stabil und performant	8: Stabil	9: Stabil
Gesamt	78.25 / 82.5 = 94.8%	53.25 / 82.5 = 64.5%	69.25 / 82.5 = 83.9%

Fazit: Obwohl Figma die höchste Gesamtbewertung erzielt, wird im Projekt Adobe XD eingesetzt, da bereits umfangreiche Erfahrung im Team vorhanden ist und es Teil der bestehenden Adobe Creative Cloud ist.

IDE / Editor

Kriterium	IntelliJ IDEA Community	Eclipse IDE	VS Code (+ Java-Extensions)
Kosten / Lizenz	Kostenlos unter Community Edition	Kostenlos, Open Source	Kostenlos
Code Intelligence (x2.0)	10: Branchenführende Analyse und vorausschauende Autovervollständigung	7: Solide Assistenz, aber oft etwas weniger intuitiv als IntelliJ	7: Gut durch „Language Support“ Plugins, aber weniger tiefgreifend
Navigation (x1.25)	9: Sehr starke Suchfunktionen „Search Everywhere“	7: Klassische Projektstruktur, Suchfunktionen sind funktional	8: Sehr schnell und effizient
Performance (x1.5)	7: Hoher RAM Verbrauch, benötigt bei großen Projekten stärkere Hardware	7: Bei vielen installierten Plugins steigende Instabilität und Hardwareanforderungen	9: Sehr leichtgewichtig und schnell startend, bei steigender Anzahl installierter Plugins ebenfalls Performance Einbrüche
Plugin-Erweiterbarkeit (x1.25)	8: Qualitativ und hochwertige Plugins	7: Riesiges Ökosystem, Plugins allerdings oft veraltet oder nicht weitergeführt	10: Eine riesige Auswahl für alles mitsamt einer sehr einfachen Installation
Built-in Features (x1.5)	9: Standardmäßig geliefert mitsamt GUI für Git und Build Tools	7: Viele Funktionen enthalten, aber oft in Untermenüs begraben	4: Minimalistisch, die meisten gewünschten Funktionen müssen per Plugin nachträglich installiert werden
Debugging und Testing (x1.75)	10: Erstklassiger Debugger und exzellente JUnit-Integration	8: Sehr mächtiger Debugger	6: Grundfunktionen vorhanden, aber weniger Komfort bei komplexeren Projekten
Scene Builder und JavaFX-Integration (x1.25)	9: Hervorragende Integration mitsamt einer einfachen Einbindung	8: Gute Unterstützung und Anbindung	4: Erfordert viel manuelle Konfiguration, sowie keinen visuellen Editor
Gesamt	94 / 105 = 89.5%	76.5 / 105 = 72.9%	71.5 / 105 = 68.3%

Fazit: Da es sich hierbei um ein Projekt überschaubarer Größe handelt, mitsamt einem eng genähten Zeitablauf, steht es den einzelnen Entwicklern frei, welche IDE sie verwenden. Ein forciert Wechsel, und die somit einhergehende Eingewöhnung in neue Entwicklungstools würden zu einer Verzögerung im Zeitplan führen.

Test-Automatisierung

Kriterium	JUnit 5	TestNG	Mockito
Verbreitung und Standard (x1.0)	10: De-facto-Standard im Java-Bereich	6: Verbreitet, aber deutlich weniger als JUnit	9: Weit verbreitet als Ergänzung zu JUnit
Einarbeitungs- aufwand (x1.5)	9: Gering bis mittel, viele Best Practices im Netz etabliert	7: Mittel, erfordert teilweise andere Denkweise	8: Gering, sofern man Architektur-Pattern kennt
Typischer Einsatz (x2.0)	10: Ideal für Unit- und Integrationstests der Spiellogik	6: Eher für komplexe Test-Szenarien und Legacy-Projekte	8: Gut für saubere Unit-Tests wie zum Beispiel Mocking
Wartbarkeit und Lesbarkeit (x1.5)	10: Sehr klare Strukturierung durch moderne Annotationen	7: Solide, kann bei großen Suites unübersichtlich werden	9: Ermöglicht lesbare Tests durch eine flüssige Schnittstelle
Build-Integration (x1.25)	10: Nahtlose Integration in Maven und gängige Pipelines	8: Gute Integration mit etwas mehr Konfigurationsaufwand	9: Problemlos in bestehende Build-Prozesse integrierbar
Eignung für Projekt (x2.0)	10: Uneingeschränkt empfohlen als Haupt-Framework	4: Möglich, aber für dieses Projekt nicht nötig	7: Sinnvoll, aber als Stand-alone-Lösung ungeeignet
Gesamt	91 / 92.5 = 98.4%	57 / 92.5 = 61.6%	75.75 / 92.5 = 81.9%

Fazit: Unsere Wahl fällt als etablierter Branchenstandard klar auf JUnit 5. Das Framework ermöglicht unserem erfahrenen Team sofortige und strukturierte Tests der Spiellogik ohne jegliche Einarbeitungshürden.

KI-Tools

Kriterium	ChatGPT Browser	GitHub Copilot	Tabnine
Integration (x1.25)	7: Nur im Browser, kein direktes IDE-Plugin	10: Hervorragend, direkt in der IDE	10: Hervorragend, direkt in der IDE
Datenschutz und Internet (x1.5)	4: Online-Zugriff zwingend, Code wird gesendet	4: Online-Zugriff zwingend, Code wird gesendet	8: Besserer Datenschutz durch teils lokales Modell
Einarbeitungs- aufwand (x1.0)	10: Sehr intuitiv dank einfacher Chat-Oberfläche	9: Gering, arbeitet einfach im Hintergrund	9: Gering, ähnliches Prinzip wie Copilot
architektonischer Nutzen (x2.0)	10: Starkes Sparring-Tool für komplexe Logik	5: Limitiert auf reine Code-Vervollständigung	5: Limitiert auf reine Code-Vervollständigung
Fehleranalyse (x1.75)	10: Exzellent für Stacktraces und tiefe Fehlerbehebung	6: Hilft primär bei lokalen Syntaxfehlern im Code	5: Kaum Unterstützung bei komplexen Laufzeitfehlern
Eignung für Projekt (x2.0)	10: Unverzichtbar für konzeptionelle Lösungsansätze	6: Optional als reine Tipphilfe	7: Optional bei starkem Fokus auf Privacy
Gesamt	82.25 / 95 = 86.6%	60 / 95 = 63.2%	66.25 / 95 = 69.7%

Fazit: Unsere Wahl fällt aufgrund der analytischen Tiefe und der vielfältigen Anwendung eindeutig auf ChatGPT. Wir benötigen fundierte Unterstützung für komplexe Architekturfragen, Systemdokumentation und tiefgreifende Fehleranalyse.

Dokumentationstools

Kriterium	Javadoc	Markdown im Repository	GitHub-Wiki
Kosten	Keine (Teil des JDK)	Keine, in GitHub	Keine
Integration im Entwicklungsprozess (x1.5)	10: Direkt aus Java-Code generierbar	9: Im selben Repo wie Code, gute Referenz	8: Web-basiert, an Repo gekoppelt
Struktur & Navigation (x1.25)	7: Klassenhierarchie automatisch	7: Eigene Links/Ordner nötig, flexibler gestaltbar	9: Seitenstruktur und Menü, Verlinkung zwischen Seiten einfach
Lesbarkeit / Darstellung (x1.25)	7: HTML-Seiten, weniger für Leser/Nutzer ohne technischen Hintergrund	8: Markdown, gut lesbar	9: Website mit Navigation, für Team gut lesbar
Kollaboration (x2.0)	6: Zusammenarbeit nur indirekt über Code, nicht parallel	8: Zusammenarbeit über Git (e.g. branches) flexibel	9: Direkt im Browser editierbar, parallel möglich
Nachvollziehbarkeit / Versionierung (x1.25)	8: Jede Änderung durch Commits sichtbar	9: Änderungen im Repo nachvollziehbar	9: Versionshistorie der Wiki-Seiten vorhanden
Aktualität / Wartbarkeit der Dokumentation (x1.5)	9: automatisch gepflegt	7: manuell, kann schnell veralten	7: manuell, kann schnell veralten
Gesamt	68 / 87.5 = 77.7%	70 / 87.5 = 80.0%	74.25 / 87.5 = 84.9%

Fazit: Für unser Projekt haben wir uns aufgrund der Anforderungen für Javadoc als primäres Dokumentationstool entschieden. Ergänzend nutzen wir Markdown für die Entwicklerdokumentation. Das Wiki von GitHub nutzen wir für Team-, Firmen- oder Ablaufinterne Vorgänge.

Obfuscator

Kriterium	ProGuard	yGuard	DashO
Build-Integration (x1.5)	10: Hervorragende und nahtlose Integration in unser Maven Setup	8: Solide Integration primär über Ant und Maven Plugins	9: Sehr gut durch eigene GUI und Build-Integration
Funktionsumfang und Optimierung (x1.25)	10: Exzellente Obfuscation kombiniert mit starkem Shrinking	7: Beschränkt auf grundlegende Obfuscation und rudimentäres Shrinking	10: Sehr umfangreiche Features inklusive tiefgehender Analyse
Konfigurationsaufwand (x1.0)	8: Moderat durch gut strukturierte Konfigurationsdateien	7: Moderat durch Konfiguration direkt über Build-Skripte	9: Eher gering dank nützlicher Assistenten
Verbreitung und Standard (x1.0)	10: Der unangefochtene Industrie-Standard für Java-Projekte	6: Bekannt, aber wesentlich weniger verbreitet in der Industrie	8: Primär in stark kommerzialisierten Großprojekten vertreten
Dokumentation und Community (x1.25)	10: Überragende Community mit unzähligen Best Practices im Netz	6: Solide Herstellerdokumentation bei sehr kleiner Community	7: Gute Herstellerdokumentation liegt oft hinter Registrierungs-schranken
Code-Sicherheit und Schutz (x2.0)	9: Bietet hohe Sicherheit für unsere klassische Java-Architektur	6: Solider Basisschutz ohne erweiterte Sicherheitsmechanismen	10: Höchste Sicherheitsstandards für extrem kritische Software
Eignung für Projekt (x2.0)	10: Uneingeschränkt empfohlen und exakt passend für unsere Architektur	4: Möglich, bietet aber gegenüber ProGuard keinerlei Vorteile	2: Kommerzieller Overkill für den geplanten Einsatzbereich
Gesamt	96 / 100 = 96.0%	61.25 / 100 = 61.3%	75.75 / 100 = 75.8%

Fazit: Unsere Wahl fällt aufgrund der nahtlosen Build-Integration und der hohen technischen Leistungsfähigkeit eindeutig auf ProGuard.

Code Conventions / Coding Style

Kriterium	Google Java Style Guide	Oracle Java Code Conventions	Spring Java Format
Lesbarkeit (x1.25)	9: Sehr klare und moderne Struktur für gut lesbaren Code	8: Verständlich, aber teilweise veraltet	9: Einheitliche Formatierung sorgt für sehr gute Lesbarkeit
Detailgrad (x1.5)	9: Sehr detailliert und umfassend	6: Eher grundlegende Regeln	7: Fokus eher auf Formatierung als auf komplette Richtlinien
Verständlichkeit / Einstieg (x1.0)	7: Viele Regeln, aber relativ gut dokumentiert	9: Einfach und leicht verständlich, viele Code-Beispiele	8: Einfach nutzbar durch automatische Formatierung
Verbreitung (x0.75)	9: Sehr verbreitet in der Praxis	7: Früher Standard, heute weniger genutzt	8: Stark verbreitet im Spring-Ökosystem
Tool-Unterstützung (x1.25)	9: Gute Integration in viele Tools	6: Weniger direkte Tool-Unterstützung	10: Stark auf automatische Formatierung ausgelegt
Flexibilität (x1.0)	6: Relativ strikt	7: Mehr Spielraum	5: Sehr strikt durch festen Formatter
Konsistenz im Code (x1.5)	10: Sehr einheitlicher Code	7: Abhängig vom Team da nur grundlegendere Regeln	10: Konsistenz wird automatisch erzwungen
Gesamt	70.75 / 82.5 = 85.8%	58.25 / 82.5 = 70.1%	68.25 / 82.5 = 82.7%

Fazit: Aufgrund dessen, dass wir eine Basis für ein modular erweiterbares Programm erstellen, haben wir uns für den Google Java Style Guide entschieden. Dieser ist modern, weitverbreitet und erzeugt aufgrund strikterer Regeln sehr einheitlichen Code.

ENTWICKLUNGS- UMGEBUNG

Kollaborationstools

Kriterium	GitHub	Discord	Miro
Hauptzweck	Versionsverwaltung, Issues, Wiki, Reviews	Chat/Sprachkanäle, schnelle Abstimmung	Planung
Kosten / Lizenz	keine	keine	keine über Hochschul-account
nachvollziehbarkeit (x1.5)	10: Projektbezogen im Repo	5: Chats teilweise ohne Zusammenhang	7: Übersichtlich am Board festgehalten
Struktur & Orga (1.75)	10: Glasklare Strukturierung	4: Eher schlecht durch konventionellen Chat	10: Top Organisation möglich
Kommunikation (x2.0)	6: Teilweise gehen Rückmeldungen verloren, etwas unübersichtlich bei z.B. vielen Pull Requests	10: Einfache, schnelle Chats, auch Anrufe möglich	2: Keine Benachrichtigung, kein richtiger Chat(verlauf), bei vielen Elementen Tendenz zu Unübersichtlichkeit
Integration (x1.0)	10: Direkt im Repo	1: Keine Integration, nur über Links und Screenshots möglich	4: Teilweise Integration über erstellten Plan möglich, sonst: Links, Screenshots
Benutzerfreundlichkeit (x0.75)	7: Konzept muss verstanden werden, Einarbeitung gut	10: Sehr einfache Bedienung, wie ein Messenger	8: Einfache Bedienung wie eine große Tafel
Zuverlässigkeit (x1.5)	10: Professionelle Infrastruktur mit hoher Uptime, Rollbacks möglich	8: Hohe Uptime, indirekter Informationsverlust	6: (Un)absichtliches Überschreiben möglich, hohe Uptime
Datenschutz / Sicherheit (x1.5)	10: Fein einstellbare Zugriffsrechte, privates Repo, Audit Log, Branch-Review System, jede Änderung sichtbar, individuelle Features vorhanden, 2FA etc.	4: Keine End to End Encryption bei normalen Chats, Zugriff über Invite-links (missbrauch möglich)	4: Weitergabe von Zugriffslink sowie u.a. Passwort möglich
Effizienz (x1.25)	8: Projektbezogenes Tracking, gute Struktur	10: Sehr schneller Austausch möglich, einfache Bedienung	2: Manuelle Pflege, hoher Aufwand
Gesamt	99.75 / 112.5 = 88.7%	73.5 / 112.5 = 65.3%	59.5 / 112.5 = 52.9%

Fazit: GitHub für nachvollziehbare, spezifisch bezogene Kommunikation, Discord für kurze Abstimmungen oder andere/generelle Fragen/Diskussionen und Anrufe. Miro bietet für uns keinen Mehrwert, da die Rahmenbedingungen schon stehen und der Aufwand zur Erstellung eines Boards nicht gerechtfertigt ist.